Design and Implementation of Scalable Database Applications with Distributed Data Management

1. Introduction and motivation

This Track is designed to bridge the gap between theoretical software design and practical system implementation. The primary objective is to build a robust, database-driven software system while gaining a deep understanding of how real-world applications handle data storage, consistency, and scalability. Students will move beyond basic application development to master the integration of advanced data structures, transaction management, and distributed systems concepts.

Structure of document:

- **Part 1: Project Phase**
  Technical project phases covering UML design, database modelling, indexing, transactions, and sharding.
- **Part 2: Evaluation**
  Assessment methodology, performance metrics, and viva-based evaluation framework.
- **Part 3: Expected outcomes**
  Functionality, design, performance, optimisation, coverage of failure scenarios.
- **Part 4: Course Project statements**
  Real-world problem domains, existing limitations, and motivation for database-driven solutions.

Instructor: Dr. Yogesh K. Meena

January 16, 2026 • Semester II (2025 - 2026) • CS 432 – Databases (Course Project)

© 2026 Indian Institute of Technology, Gandhinagar. All rights reserved.

2. Project Phase

Assignment 1: Database and UML Development

This assignment involves conducting a detailed requirement analysis of the selected problem statement to clearly define system scope and functionality. A system architecture will be designed using standard UML diagrams to model structural and behavioural aspects of the application. Based on this design, a normalised relational database schema will be developed, specifying entities, relationships, primary and foreign keys, and integrity constraints to ensure an efficient and consistent data model.

Learning Objectives:

- System Modelling: Transforming real-world requirements into formal technical models.
- Architectural Design: Understanding data and control flow within a complete software system.
- Database Normalisation: Applying principles from 1NF to 3NF to reduce redundancy and dependencies.
- Schema Mapping: Converting object-oriented class diagrams into relational database tables.

Assignment 2: Application and B+ Tree Development

This assignment requires the development of core backend application logic along with a custom B+ Tree indexing structure implemented from scratch. The B+ Tree will function as a high-performance indexing layer for critical attributes such as User IDs, Order Timestamps, or Transaction Logs. The application must integrate this index to support efficient search and range query operations, demonstrating the practical impact of data structures on system performance.

Learning Objectives:

- Indexing Internals: Understanding how database indexes enable fast data retrieval.
- Algorithmic Implementation: Implementing B+ Trees to support logarithmic-time operations.
- Query Optimisation: Demonstrating performance gains of indexed access over linear scanning.
- System Integration: Linking low-level data structures with high-level application logic.

Assignment 3: ACID Testing on the Application and B+ Tree

This assignment focuses on evaluating the system under concurrent workloads to validate ACID properties (Atomicity, Consistency, Isolation, Durability). Simulated multi-user scenarios will be created to test simultaneous read, write, and update operations. Special attention will be given to maintaining consistency between the main database and the custom B+ Tree index during transaction rollbacks, crashes, or partial failures.

Learning Objectives:

- Transaction Management: Designing operations that either complete fully or revert safely.
- Concurrency Control: Applying locking and synchronisation mechanisms to prevent conflicts.
- Data Integrity: Ensuring alignment between database records and index structures.
- Crash Recovery: Studying mechanisms that guarantee data persistence after failures.

Assignment 4: Sharding of the Developed Application

This assignment involves implementing logical data partitioning (sharding) across multiple simulated nodes or tables. A suitable Shard Key (such as Region, User ID, or Hash value) will be selected, and application logic will be modified to correctly route queries to the appropriate shard. The exercise simulates real-world distributed database systems that scale horizontally to handle large datasets.

Learning Objectives:

- Horizontal Scaling: Distinguishing between vertical and horizontal system expansion.
- Sharding Strategies: Applying range-based, directory-based, and hash-based partitioning.
- Distributed Query Routing: Locating data across multiple partitions efficiently.
- Scalability Trade-offs: Analysing consistency, availability, and partition tolerance in distributed systems.

3. Evaluation Plan

The proposed system will be evaluated along the following dimensions:

- **Correctness and Design Quality:** Accuracy of requirement analysis, UML modelling, schema normalisation, and alignment between application logic and database design.
- **Indexing and Transactional Behaviour:** Proper implementation of the B+ Tree indexing structure and validation of ACID properties under concurrent access and failure conditions.

- **Scalability and Performance:** Impact of indexing and sharding on query latency, throughput, and system overhead compared to baseline approaches.

Evaluation will follow predefined rubrics for each assignment, with a viva conducted after every assignment.

Note: Separate documents will be shared for each assignment and evaluation.

4. Expected Outcomes

By the end of Track 1, the project is expected to achieve the following outcomes:

- A well-structured system design based on requirement analysis
- UML modelling, and normalised relational schemas.
- A functional application
- A custom B+ Tree indexing mechanism for efficient data retrieval.
- A robust system demonstrating ACID compliance under concurrent and failure scenarios.
- A sharded data architecture illustrating principles of distributed databases and horizontal scalability.
- A scalable database-driven application capable of handling real-world complexities.